MODULE 4

DAY 2 · MODULE 4 OF 8

From Clarified Spec to Reviewable Plan

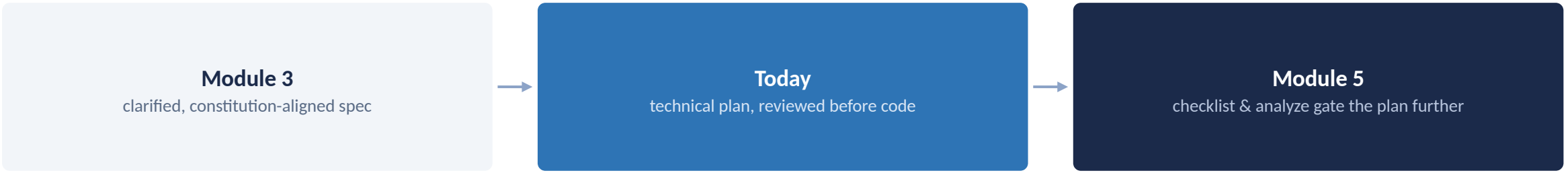
Turn a what-and-why spec into a reviewable how, before any code changes.

What You'll Learn

1. Translate a clarified spec into a technical plan
2. Cover architecture choices, constraints, and non-functional requirements
3. Use a plan/preview mode to review before any files change
4. Decompose a plan into small, independently verifiable tasks
5. Recognize what a review at plan time catches that a code review can't undo cheaply

WHY THIS MATTERS

Module 3 gave you a clarified what-and-why, today it becomes a reviewable how.



Day 2 starts here: everything from here forward assumes a clarified spec already exists

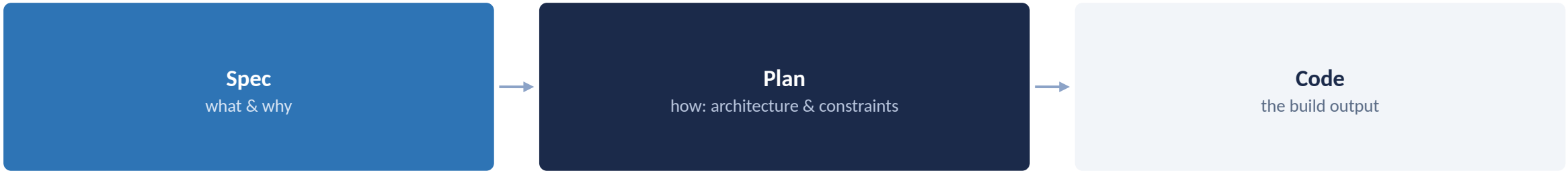
PART 1

From Spec to Plan

Where implementation detail belongs, and what a good plan actually covers

FROM SPEC TO PLAN

The Plan Is the Bridge Between Why and How



The plan is the bridge between why and how

MODULE 4

REAL-WORLD CONTEXT

A Good Plan Covers Three Things, Beyond Just Architecture

Architecture choices
how components fit together

Constraints
technical, organizational, whatever already applies

Non-functional requirements
performance, security, scale expectations

Skipping non-functional requirements is how a plan passes review and still needs a rewrite

CHECKPOINT

Let's Pause and Verify

1 What three things does a good technical plan cover, besides architecture choices?

2 Where does implementation detail belong, the spec or the plan?

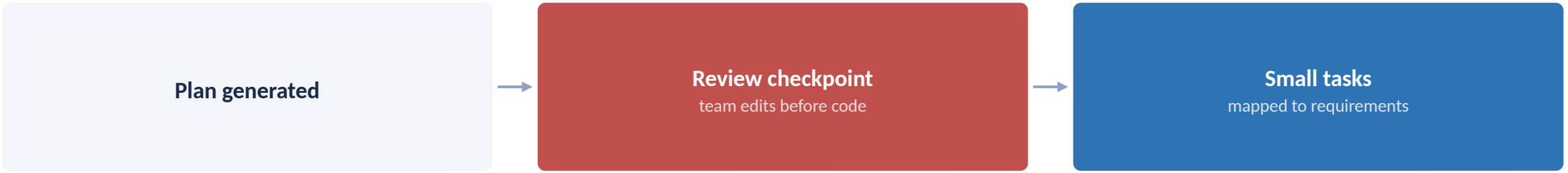
PART 2

Review Before Code

Plan/preview mode as a deliberate stop before any file changes

REVIEW BEFORE CODE

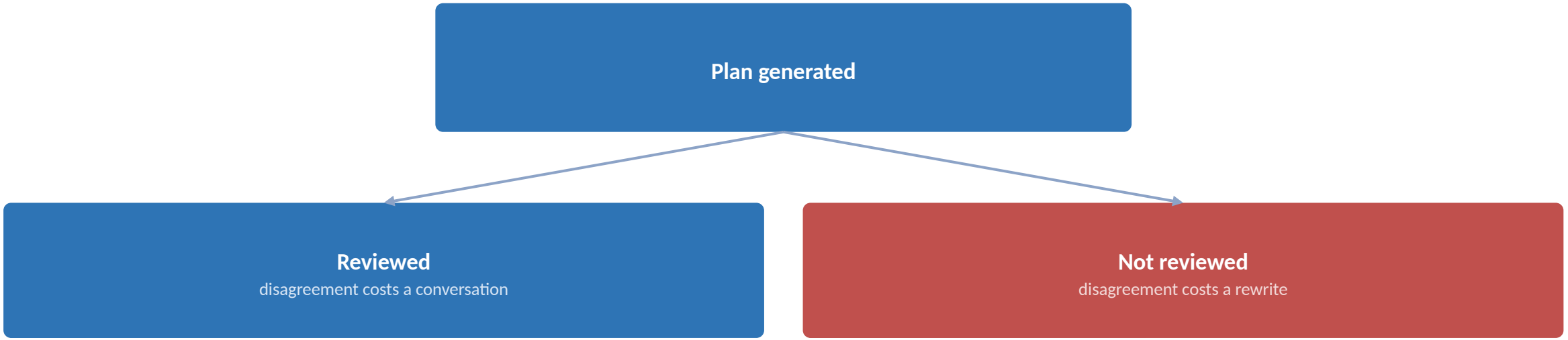
A Deliberate Stop Before Any File Changes



A deliberate stop before any file changes

WHY IT MATTERS

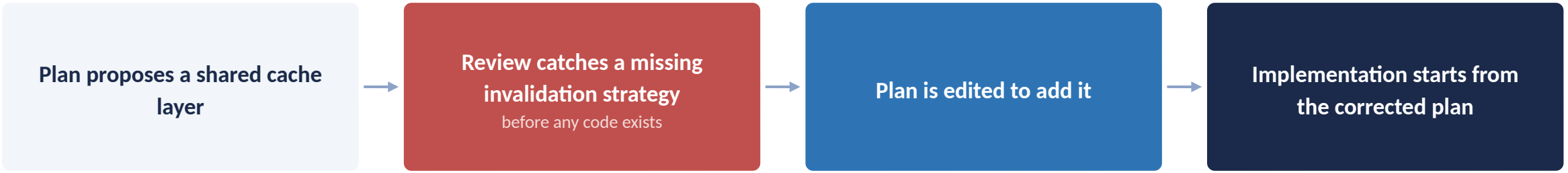
A Plan You Didn't Review Is a Decision You Already Made



Silence during plan review is implicit approval

REAL-WORLD CONTEXT

This Is What Catching It Early Actually Looks Like



The correction happened in a document, not in a pull request three days later

CHECKPOINT

Let's Pause and Verify

1 Why is silence during plan review the same as approval?

2 Name one kind of gap worth catching at plan-review time rather than after code exists.

PART 3

Breaking the Plan Into Work

Small, independently verifiable, agent-executable tasks

A Well-Decomposed Task Has Four Properties

Small
reviewable in one sitting

Independently verifiable
can be checked without finishing the rest

Maps to a requirement
traceable back to the spec

Agent-executable
clear enough to act on without further clarification

A task missing any one of these four properties is a plan that isn't finished decomposing yet

WHERE THIS FITS

Spec Kit Separates Plan and Tasks, OpenSpec Generates Them Together

both support a staged review, just structured differently

Neither framework skips the review checkpoint, they just place it differently

CHECKPOINT

Let's Pause and Verify

1 Name one property of a well-decomposed task, besides being small.

2 Which framework generates the plan and tasks together, and which keeps them as separate commands?

MODULE 4

COMMON PITFALLS

A task bundling five requirements is Module 1's unreviewable diff, one level down.

the failure mode is the same, just moved one layer down the lifecycle

If a task's diff needs its own paragraph to explain what's in scope, it's not decomposed yet

Key Takeaways

1. The plan translates a clarified spec's what-and-why into a reviewable how: architecture, constraints, non-functional requirements.

2. A plan you haven't reviewed is a plan you're implicitly approving, whether or not that was your intent.

3. Catching a bad architecture decision at plan-review time costs a conversation; catching it after code exists costs a rewrite.

4. Small, independently verifiable, requirement-mapped tasks are what make a diff reviewable, this is Module 1's failure mode, solved at the task level.

5. Next: Module 5, Quality Gates Before Implementation, adds checklist and analyze as the last checks before any of these tasks get implemented.